Software Requirements Specification

for

Sequel Speak

Schema-Aware Text-to-SQL System

Version 1.0

January 4, 2026

Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for **Sequel Speak**, a schema-aware text-to-SQL system. The system translates natural language queries into valid SQL statements, executes them against databases, and presents results intelligently.

This document is intended for:

- Development team for system implementation
- Academic evaluators for project assessment
- QA team for test case derivation

1.2 Document Conventions

- **FR-XX**: Functional Requirement identifier
- **NFR-XX**: Non-Functional Requirement identifier
- `Monospace`: Code, API endpoints, technical terms

1.3 Product Scope

Sequel Speak enables non-technical users to query databases using plain English while providing transparency for technical users.

Core Capabilities:

- Natural language to SQL translation using LLM
- Dynamic database schema extraction and indexing
- Semantic schema search using vector embeddings
- SQL validation and secure query execution
- Query history and user feedback collection
- RESTful API and web-based interface

1.4 References

1. IEEE Std 830-1998: Software Requirements Specifications
2. OpenRouter API: https://openrouter.ai/docs
3. PostgreSQL: https://www.postgresql.org/docs/
4. pgvector: https://github.com/pgvector/pgvector
5. FastAPI: https://fastapi.tiangolo.com/

2. Overall Description

2.1 Product Perspective

Sequel Speak is a standalone web application with a FastAPI backend and responsive frontend, backed by PostgreSQL with pgvector extension for semantic search.

2.1.1 System Architecture

The system follows a **10-step modular pipeline**:

1. **Schema Connector** – Extract database metadata
2. **Schema Indexer** – Store schema with vector embeddings
3. **Intent Understanding** – Parse natural language query
4. **Context Retriever** – Retrieve relevant schema elements
5. **SQL Generator** – Generate SQL using LLM
6. **Validator** – Check syntax and security
7. **Executor** – Run validated SQL securely
8. **Result Presenter** – Format output with explanations
9. **Feedback Loop** – Capture user corrections
10. **Learning System** – Update embeddings from feedback

2.2 Product Functions

- Accept natural language queries and generate SQL
- Extract and cache database schema automatically
- Validate SQL syntax and block dangerous operations
- Execute queries with result limiting
- Display generated SQL alongside results (transparency)
- Store query history and collect user feedback
- Provide API documentation via Swagger UI

2.3 User Classes

User Class	Description	Proficiency
Business Analyst	Queries databases for reports using natural language	Low-Medium
Data Scientist	Explores data, validates and refines generated SQL	Medium-High
DBA	Manages connections, refreshes schemas	High
Developer	Extends functionality, creates connectors	Expert

2.4 Operating Environment

- **Server**: Python 3.10+, 4GB RAM minimum, Linux/Windows/macOS
- **Client**: Modern browsers (Chrome 90+, Firefox 88+, Safari 14+)
- **Database**: PostgreSQL 12+ with pgvector extension
- **Network**: Internet access for OpenRouter API

2.5 Design Constraints

- Schema agnostic – never hard-code table/column names
- Dynamic reflection – always query live schema
- Safety first – validate before execution
- Transparency – always show generated SQL to users
- PostgreSQL first – other dialects are future scope

2.6 Dependencies

- FastAPI (≥0.104.0) – Web framework
- SQLAlchemy (≥2.0.0) – ORM
- psycopg (≥3.1.0) – PostgreSQL driver
- pgvector (≥0.2.0) – Vector similarity search
- OpenAI SDK (≥1.0.0) – LLM API client
- sentence-transformers (≥2.2.0) – Embeddings

3. External Interface Requirements

3.1 User Interfaces

The web interface includes:

- **Database Selector**: Dropdown for saved connections
- **Query Input**: Textarea for natural language queries
- **SQL Display**: Syntax-highlighted generated SQL
- **Results Table**: Dynamic table with query results
- **Loading Indicator**: Status during processing
- **Notification System**: Toast messages for feedback

API documentation available at `/docs` (Swagger) and `/redoc`.

3.2 Software Interfaces

Interface	Details
OpenRouter API	HTTPS REST, Bearer auth, default model: claude-3.5-sonnet
PostgreSQL	psycopg3 driver, connection pooling, pgvector extension
Embedding Model	sentence-transformers/all-MiniLM-L6-v2 (384 dimensions)

3.3 Communications Interfaces

- REST API: HTTP/HTTPS for frontend-backend communication

- Database: TCP connections to PostgreSQL (port 5432)
- LLM API: HTTPS calls to OpenRouter

4. Functional Requirements

4.1 Database Connection Management

FR-1. The system shall accept PostgreSQL connection URLs in standard format.

FR-2. The system shall URL-encode special characters in passwords automatically.

FR-3. The system shall test connectivity and return success/failure status.

FR-4. The system shall store connection profiles in browser LocalStorage.

4.2 Schema Extraction and Indexing

FR-5. The system shall extract table names, column names, and data types.

FR-6. The system shall extract primary and foreign key relationships.

FR-7. The system shall cache schema with configurable TTL (default: 30 min).

FR-8. The system shall provide manual schema refresh via API.

FR-9. The system shall generate vector embeddings for schema elements.

FR-10. The system shall store embeddings in pgvector for similarity search.

4.3 Natural Language Query Processing

FR-11. The system shall accept natural language queries in English.

FR-12. The system shall provide relevant schema context to LLM.

FR-13. The system shall use configurable LLM model via OpenRouter API.

FR-14. The system shall generate PostgreSQL-compatible SQL syntax.

FR-15. The system shall clean LLM output by removing markdown blocks.

FR-16. The system shall classify query intent before SQL generation.

4.4 SQL Validation and Security

FR-17. The system shall validate SQL syntax using sqlparse.

FR-18. The system shall block DROP, DELETE, TRUNCATE, ALTER by default.

FR-19. The system shall detect and block SQL injection patterns.

FR-20. The system shall validate table/column references against schema.

FR-21. The system shall allow dangerous operations with explicit confirmation.

4.5 Query Execution and Results

FR-22. The system shall execute SQL against connected PostgreSQL database.

FR-23. The system shall limit result rows to configurable maximum (default: 100).

FR-24. The system shall return results as JSON with column names and values.

FR-25. The system shall display generated SQL alongside results.

FR-26. The system shall measure and report query execution time.

FR-27. The system shall implement query execution timeout (30 seconds).

4.6 Query History

FR-28. The system shall store all executed queries with metadata.

FR-29. The system shall record NL query, generated SQL, and status.

FR-30. The system shall record execution time and row count.

FR-31. The system shall provide API to retrieve query history.

4.7 Feedback and Learning

FR-32. The system shall accept SQL corrections from users.

FR-33. The system shall accept star ratings (1-5) for queries.

FR-34. The system shall store feedback linked to original query.

FR-35. The system shall update embeddings based on feedback.

5. Non-Functional Requirements

5.1 Performance

NFR-1. End-to-end response time shall be less than 5 seconds.

NFR-2. Schema extraction (up to 100 tables) shall complete in under 3 seconds.

NFR-3. Schema cache hits shall respond in under 50ms.

NFR-4. API shall handle 50+ concurrent requests.

NFR-5. Frontend load time shall be under 2 seconds.

5.2 Security

NFR-6. The system shall prevent SQL injection attacks.

NFR-7. The system shall block dangerous SQL operations by default.

NFR-8. The system shall never log passwords in plain text.

NFR-9. The system shall implement rate limiting.

NFR-10. The system shall sanitize LLM prompts against injection.

5.3 Reliability

NFR-11. System uptime shall target 99.5% availability.

NFR-12. The system shall degrade gracefully when OpenRouter is unavailable.

NFR-13. The system shall provide health check endpoint at `/health`.

NFR-14. The system shall support automatic database reconnection.

5.4 Usability

NFR-15. New users shall execute first query within 5 minutes.

NFR-16. Error messages shall be human-readable with suggestions.

NFR-17. API documentation shall be auto-generated and current.

NFR-18. UI shall be responsive across desktop and tablet devices.

5.5 Maintainability

NFR-19. Code shall follow PEP 8 style guidelines.

NFR-20. All functions shall have type hints.

NFR-21. Test coverage shall exceed 80% for core modules.

NFR-22. LLM prompts shall be in separate files under `prompts/`.

NFR-23. Configuration shall use environment variables.

5.6 Scalability

NFR-24. The system shall support databases with up to 500 tables.

NFR-25. The system shall support horizontal scaling via containers.

NFR-26. Database connections shall use connection pooling.

6. Appendix

6.1 Data Entities

SchemaEmbedding:

- `id` – Primary key
- `table_name`, `column_name`, `data_type`
- `embedding` – Vector(384)
- `created_at` – Timestamp

QueryHistory:

- `id` – UUID primary key
- `nl_query`, `generated_sql`
- `execution_time`, `success`, `row_count`
- `created_at` – Timestamp

Feedback:

- `id` – UUID primary key
- `query_id` – Foreign key to QueryHistory
- `feedback_type`, `corrected_sql`, `rating`
- `created_at` – Timestamp

6.2 API Endpoints

Method	Endpoint	Description
GET	`/`	API information
GET	`/health`	Health check
POST	`/api/query/execute`	Execute NL query
GET	`/api/query/history`	Get query history
POST	`/api/schema/refresh`	Refresh schema
GET	`/api/schema/tables`	List tables
POST	`/api/schema/search`	Semantic search
POST	`/api/feedback/submit`	Submit feedback
POST	`/api/utils/test-connection`	Test DB connection

End of Document

Sequel Speak – SRS v1.0 — Cognic AI Team